

Using MathCompile

MathCompile is a worksheet of math cells. Type math on the left, and the output column on the right gives you the result in the format you pick: LaTeX source, computed answers, or SymPy Python code. It runs entirely in your browser and saves your worksheet automatically.

The worksheet at a glance

- **Cells** are numbered rows. Click a cell to edit it; the up and down arrow keys move between cells when the caret reaches an edge. **+ Add expression** appends a cell, and **×** deletes one. Pressing Backspace in a cell that holds only blank space deletes it too.
- **The output column** shows whatever the current output target produces for each cell. Drag the divider between the columns to resize it.
- **The header** holds Smart mode, the Output target picker, the dark-mode toggle, and the Commands button (or press `Ctrl+K` / `⌘K`).
- **How do I...**, under the cell list, is a quick keystroke cheat sheet. This document is the long version.

The command palette (`Ctrl+K`) covers: insert an expression below, duplicate the current one, delete it, clear all expressions, toggle Smart mode and dark mode, switch the output target, and jump to any cell by number.

Typing math

Smart mode (the default)

With Smart mode on — the checkbox in the header — you rarely need a backslash. Whole words convert to symbols as you type, and a digit right after a letter becomes a subscript:

You want	Type	Renders
Definite integral	<code>int</code>	$\int$ with upper and lower bound blocks
Indefinite integral	<code>iint</code> or <code>antid</code>	a bare $\int$
Sum / product	<code>sum</code> , <code>prod</code>	Σ , Π with bound blocks
Square root	<code>sqrt</code>	$\sqrt{\quad}$
Constants and Greek	<code>pi</code> , <code>theta</code> , <code>infty</code>	π , θ , ∞
Derivative operator	<code>derivative</code>	$\frac{d}{dx}$ — caret lands in the denominator
Function definition	<code>def</code>	def $f(x)$ — the def marker (see <i>Output</i>)
Auto-subscript	<code>x2</code>	x_2

Operator names like `sin` , `lim` , and `det` stay upright and space correctly without any command.

LaTeX commands

Type `\` followed by a command name — `\frac`, `\sqrt`, `\int`, `\alpha` — and press **Enter** or **Space** to accept it. As you type, a suggestion menu lists matching commands; the arrow keys highlight and Enter accepts the highlight, or just keep typing to filter. `Esc` dismisses the menu. Word-runs of two or more letters suggest too (`sq` offers `\sqrt`).

`Ctrl+Space` opens the searchable symbol picker: type to filter, Enter inserts, `Esc` closes.

Unicode characters insert directly — typing `α`, `≤`, `∫`, `∇` just works.

Common things you'll type

You want	Type	Renders
Fraction	<code>1/2</code> , or <code>\frac</code> then fill the blocks	$\frac{1}{2}$
Subscript / superscript	<code>_</code> , <code>^</code> (or <code>x2</code> in Smart mode)	x_1, x^2
Square / nth root	<code>sqrt</code> , or <code>\nthroot</code>	$\sqrt{x}, \sqrt[n]{x}$
Sum / product with bounds	<code>sum</code> or <code>prod</code> , fill the bound blocks	$\sum_{k=1}^n, \prod_{k=1}^n$
Definite integral	<code>int</code> , fill the bound blocks	$\int_0^1$
Indefinite integral	<code>iint</code> or <code>antid</code> — add bounds later with <code>_</code>	$\int$
Limit	<code>\lim</code> , then <code>_ {x\to0}</code>	$\lim_{x\rightarrow 0}$
Binomial coefficient	<code>\binom</code>	$\binom{n}{k}$
Matrix	<code>\pmatrix</code> — Enter adds a row, <code>Shift+Space</code> a column	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$
Piecewise	<code>\cases</code>	$\begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$
Greek letters	<code>\alpha</code> , <code>\beta</code> , ... or <code>pi</code> , <code>theta</code> , <code>infty</code>	α, β, π
Text inside math	<code>\text{...}</code>	rate
Accents	<code>\dot</code> , <code>\ddot</code> , <code>\bar</code> , <code>\hat</code> , <code>\vec</code> , <code>\tilde</code>	$\dot{x}, \hat{x}, \vec{v}$
Sets	<code>\in</code> , <code>\notin</code> , <code>\subseteq</code> , <code>\cup</code> , <code>\cap</code> , <code>\emptyset</code> , <code>\reals</code>	$x \in A, A \cup B, \emptyset, \mathbb{R}$
Relations	<code>\leq</code> , <code>\geq</code> , <code>\neq</code> , <code>\approx</code> , <code>\pm</code>	$a \leq b, a \neq b, a \pm b$
Arrows	<code>\to</code> , <code>\rightarrow</code> , <code>\leftrightarrow</code>	$x \rightarrow 0, p \Rightarrow q, p \Leftrightarrow q$
Calculus	<code>\partial</code> , <code>\nabla</code> , <code>\infty</code> , <code>\prime</code>	$\partial, \nabla, \infty, f'$
Logic	<code>\forall</code> , <code>\exists</code> , <code>\therefore</code> , <code>\land</code> , <code>\lor</code> , <code>\lnot</code>	$\forall x, \exists x, \therefore, \neg p$

`SYMBOLS.md` in the repository lists every command the editor accepts.

Getting in and out of blocks

Commands like `\frac` open placeholder blocks, and the caret stays inside the block you're filling. Everything you type lands there — including `+`, `=`, and spaces. To move on:

- **Right** or **Tab** steps out of the current block.
- `(` `[` `{` insert closed pairs with the caret inside; typing the closer again hops over it. `\angle` pairs with `\rangle`.
- Backspace clears the block you're in before it removes the command that owns it.

Multi-line cells and cells

Enter starts a new line inside the same cell (useful for aligned derivations); **Shift+Enter** starts a whole new cell. While a `\command` input is still open, Enter accepts the command instead of breaking a line.

Linear algebra

Environments open two ways: type `\name` directly (the shortcut) or `\begin{name}` the long way. In the table, each entry shows the shortcut and what a `a,b / c,d` grid renders as.

Matrices

You want	Type	Renders
Parenthesized matrix	<code>\pmatrix</code>	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$
Bracketed matrix	<code>\bmatrix</code>	$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$
Braced matrix	<code>\Bmatrix</code>	$\begin{Bmatrix} a & b \\ c & d \end{Bmatrix}$
Barred matrix	<code>\vmatrix</code>	$\begin{vmatrix} a & b \\ c & d \end{vmatrix}$
Double-barred matrix	<code>\Vmatrix</code>	$\begin{Vmatrix} a & b \\ c & d \end{Vmatrix}$
Bare grid	<code>\matrix</code>	$\begin{matrix} a & b \\ c & d \end{matrix}$
Compact inline matrix	<code>\smallmatrix</code>	$\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}$

Inside a matrix grid: **Enter** adds a row, **Shift+Space** adds a column, and the arrow keys walk cell to cell. `&` separates columns and `\\` ends a row in the stored LaTeX. The bare forms `\pmatrix{a&b\\c&d}` and friends also work — one brace argument, no `\begin`.

Matrix operations

`det` is an upright name (works with or without Smart mode); the rest of the ops go through `\mathrm{name}` — the compiler reads upright names as functions, so these become real matrix methods in the Calculator and Python targets. A name that isn't built in, like `\rank`, types as plain `\text{rank}` — not an operator.

Operation	Type	Renders
Determinant	<code>det</code> then the matrix	$\det A$
Trace	<code>\mathrm{trace}</code> (or <code>\mathrm{tr}</code>)	$\mathrm{trace}(A)$
Rank	<code>\mathrm{rank}</code>	$\mathrm{rank}(A)$
Inverse	<code>A^{-1}</code> or <code>\mathrm{inverse}</code>	A^{-1}
Transpose	<code>A^{\mathrm{T}}</code> or <code>\mathrm{transpose}</code>	A^T
Norm	<code>\lVert</code> (auto-pairs)	$\ A\ $
Eigenvalues	<code>\mathrm{eigenvals}</code>	$\mathrm{eigenvals}(A)$
Eigenvectors	<code>\mathrm{eigenvects}</code>	$\mathrm{eigenvects}(A)$
Kernel / null space	<code>\ker</code>	$\ker A$

Multi-line equations

You want	Type	Renders
Centered stack of equations	<code>\gathered</code>	$ \begin{array}{c} x + y = 1 \\ x - y = 3 \end{array} $
Equations aligned on <code>=</code>	<code>\aligned</code>	$ \begin{array}{l} x = y \\ a = b \end{array} $
Piecewise function	<code>\cases</code>	$ \begin{cases} x^2 & x > 0 \\ 0 & x \leq 0 \end{cases} $

Aliases open the same grid: `\gather`, `\equation`, `\multline`, and `\flalign` all open a centered stack like `\gathered`; `\align` and `\split` open an aligned grid. `\begin{...}` additionally accepts `eqnarray` and `eqalign` (also aligned grids) — those two have no `\name` shortcut. Rows and cells edit like matrices: Enter adds a row, `&` separates columns.

Custom grids

Grids that take a column spec or a count open a pending `{ }` block when you type the `\name` — enter the argument (`cc`, `l|r`, `2`) and press `}`, Enter, or Tab to apply it.

You want	Type	Renders
Grid with a column spec	<code>\array{lr}</code>	$ \begin{array}{lr} a & b \\ cc & d \end{array} $
Spec'd grid for text	<code>\tabular{ll}</code>	a spec'd grid like <code>\array</code> , for text content
Stacked limit labels	<code>\subarray{c}</code>	$ \subarray{c} \begin{array}{c} a \\ b \end{array} $
Column pairs aligned together	<code>\alignat{2}</code> / <code>\alignedat{2}</code>	$ \begin{array}{ll} x = y & u = v \\ a = b & c = d \end{array} $
Compact vertical stack	<code>\substack</code>	$ \substack{a \\ b} $

Set theory

The named sets have both a letter command and a word command (several spellings too — `\reals` / `\Reals`, `\integers` / `\Integers`, `\complex` / `\complexes` / `\complexplane`, `\rationals`, `\naturals`, `\quaternions`, `\primes`). Spelling out `\mathbb{R}` doesn't resolve as a command; use one of these instead.

You want	Type	Renders
Real numbers	<code>\R</code> or <code>\reals</code>	$\mathbb{R}$
Integers	<code>\Z</code> or <code>\integers</code>	$\mathbb{Z}$
Rational numbers	<code>\Q</code> or <code>\rationals</code>	$\mathbb{Q}$
Complex numbers	<code>\C</code> or <code>\complexes</code>	$\mathbb{C}$
Natural numbers	<code>\N</code> or <code>\naturals</code>	$\mathbb{N}$
Quaternions	<code>\H</code> or <code>\quaternions</code>	$\mathbb{H}$
Primes / projective	<code>\P</code> or <code>\primes</code>	$\mathbb{P}$
Integers mod n	<code>\Z</code> or <code>\integers</code> , then <code>_n</code>	$\mathbb{Z}_n$
Membership	<code>x \in A</code> , <code>\notin</code>	$x \in A, x \notin A$
Union / intersection / difference	<code>\cup</code> <code>\cap</code> <code>\setminus</code>	$A \cup B, A \cap B, A \setminus B$
Subset	<code>\subset</code> <code>\subseteq</code> <code>\nsupseteq</code>	$A \subset B, A \subseteq B, A \not\subseteq B$
Empty set	<code>\emptyset</code> or <code>\varnothing</code>	$\emptyset$
Finite set	<code>\{1,2,3\}</code>	$\{1,2,3\}$ — braces auto-pair
Set-builder	<code>\{ x \in \mathbb{R} \mid x > 0 \}</code>	$\{x \in \mathbb{R} \mid x > 0\}$
Indexed union / intersection	<code>\bigcup</code> <code>\bigcap</code> , then <code>_i</code>	$\bigcup_{i=1}^n A_i, \bigcap_{i=1}^n A_i$
Interval	<code>(a,b)</code> , <code>[a,b)</code> , <code>[a,b]</code>	$(a,b), [a,b), [a,b]$ — parens/brackets auto-pair

These compile to real SymPy sets: `x \in \mathbb{R}` puts `real=True` on `x`'s symbol (likewise `\Z` → integer, `\Q` → rational, `\C` → complex), `\cup` / `\cap` / `\setminus` become `Union` / `Intersection` / `Complement`, `\{1,2,3\}` a `FiniteSet`, `\{x \mid p(x)\}` a `ConditionSet`, and `\{f(x) \mid x \in D\}` an `ImageSet`. `A \subseteq B` needs concrete sets on both sides — SymPy can't name an unknown set, so bare symbols get an ! error note.

Logic

You want	Type	Renders
Connectives	<code>\land</code> <code>\lor</code> <code>\neg</code>	$p \wedge q, p \vee q, \neg p$
Implication / equivalence	<code>\implies</code> <code>\iff</code>	$p \implies q, p \iff q$
Quantifiers	<code>\forall</code> <code>\exists</code>	$\forall x \in S, \exists x \in S$
Turnstile / satisfaction	<code>\vdash</code> <code>\models</code>	$T \vdash p, M \models p$
True / false	<code>\top</code> <code>\bot</code>	$\top, \bot$

You want	Type	Renders
Therefore / because	<code>\therefore</code> <code>\because</code>	$\therefore$ $\because$

Connectives lower to `And` / `Or` / `Not` / `Implies` / `Equivalent`. A quantifier over a concrete set folds elementwise (`\forall x \in \{1,2,3\}` becomes an `And` over the elements); SymPy has no general quantifiers, so an abstract `\forall` is flagged rather than computed.

Calculus

`\int`, `\sum`, and `\prod` come with `_` / `^` bound blocks already made — type the bound, then `Right` or `Tab` to the next block. A typed `_` / `^` inside a bound nests a block instead of moving. `\lim` has no pre-made bound: type `{x \to 0}` yourself.

You want	Type	Renders
Definite integral	<code>\int</code> , bounds <code>0</code> then <code>1</code> , <code>x^2</code> <code>Right</code> <code>dx</code>	$\int_0^1 x^2 dx$
Indefinite integral	<code>\antid</code>	$\int f dx$ — adds $+C$ in the calculator
Double / triple integral	<code>\iint</code> <code>\iiint</code>	$\iint, \iiint$ — boundless signs
Closed integral	<code>\oint</code> <code>\oiint</code>	$\oint, \oiint$
Derivative	<code>\frac{d}{dx}</code> or <code>\derivative</code>	$\frac{dy}{dx}$
Prime / nth derivative	<code>f'</code> , <code>f''</code> , <code>f^{(n)}</code>	$f'(x), f''(x), f^{(n)}(x)$
Sum / product	<code>\sum</code> then <code>k=1</code> <code>Right</code> <code>n</code> ; <code>\prod</code> same	$\sum_{k=1}^n a_k, \prod_{k=1}^n a_k$
Limit, incl. one-sided	<code>\lim</code> then <code>{x \to 0}</code> ; <code>0^{+}</code>	$\lim_{x \rightarrow 0} f(x), \lim_{x \rightarrow 0^+} f(x)$
Lim sup / lim inf	<code>limsup</code> , <code>liminf</code>	lim sup , lim inf — upright operator names
Infinity	<code>\infty</code>	∞

Integrals, derivatives, sums, products, and limits all compile to the real SymPy calls — `integrate`, `diff`, `Sum` / `Product`, `limit` (one-sided included).

Multivariable calculus

You want	Type	Renders
Partial symbol	<code>\partial</code>	∂
Partial derivative	<code>\frac{\partial u}{\partial t}</code>	$\frac{\partial u}{\partial t}$
Partial derivative, shorter	<code>\partial</code> <code>_</code> <code>x</code> then <code>u</code>	$\partial_x u$ — reads as the same derivative
Iterated integral	<code>\iint</code> , <code>\iiint</code> , then <code>f</code> <code>dx dy</code>	$\iint f dx dy, \iiint f dx dy dz$
Closed surface integral	<code>\oiint</code>	$\oiint$
Gradient / Laplacian notation	<code>\nabla</code>	$\nabla f, \nabla^2 u$
Laplacian symbol	<code>\Delta</code>	Δu
Vector	<code>\vec{v}</code>	$\vec{v}$

`\partial_{x}u` compiles to the partial derivative like the `\frac` form. `\nabla` and `\Delta` are display notation — the compiler reads `\nabla a` as an ordinary symbol, so `\nabla f` stays symbolic rather than becoming a gradient call.

Topology

You want	Type	Renders
Map between spaces	<code>f : X \to Y</code>	$f : X \rightarrow Y$
Element mapping	<code>\mapsto</code>	$x \mapsto x^2$
Homeomorphic	<code>\cong</code>	$X \cong Y$
Product space	<code>\times</code>	$X \times Y$
Closure	<code>\bar{A}</code> or <code>\overline{A}</code>	$\overline{A}$
Boundary	<code>\partial A</code>	∂A
Interior	<code>A^{\circ}</code>	A°
Complement	<code>A'</code>	A'
Open / half-open sets	<code>(a,b)</code> , <code>[a,b)</code>	$(a,b), [a,b)$
Neighborhood eps / delta	<code>\varepsilon</code> <code>\delta</code>	ε, δ

Topology rows are notation — unions, intersections, closures, and maps display but carry no topological semantics. One exception to know: `\cong` is read by the compiler as modular congruence ($x \cong b \pmod m \rightarrow \text{Eq}(\text{Mod}(x, m), b)$), not isomorphism.

Complex analysis

You want	Type	Renders
Complex numbers	<code>\mathbb{C}</code> or <code>\complexes</code>	$\mathbb{C}$
Real / imaginary part	<code>\Re</code> <code>\Im</code>	$\Re z, \Im z$
Argument	<code>\arg</code>	$\arg z$
Conjugate	<code>\bar{z}</code> or <code>\overline{z}</code>	$\bar{z}$
Modulus	<code> z </code>	$ z $ — <code> </code> auto-pairs
Polar form	<code>e^{i\theta}</code>	$e^{i\theta}$
Contour integral	<code>\oint</code>	$\oint_{\gamma} f(z) dz$
Weierstrass p / script ell	<code>\wp</code> <code>\ell</code>	$\wp, \ell$

`\Re` , `\Im` , `\arg` , and `\overline{z}` compile to `re` , `im` , `arg` , and `conjugate` ; `i` is the imaginary unit.

Abstract algebra

You want	Type	Renders
Integers mod n	<code>\Z</code> or <code>\integers</code> , then <code>_n</code>	$\mathbb{Z}_n$
Generated subgroup	<code>\langle a \rangle</code>	$\langle a \rangle$ — auto-pairs
Product / direct sum / tensor	<code>\times</code> <code>\oplus</code> <code>\otimes</code>	$G \times H, G \oplus H, G \otimes H$
Composition	<code>\circ</code>	$f \circ g$
Generic operation	<code>\star</code>	$a \star b$
Kernel of a map	<code>\ker</code> <code>\phi</code>	$\ker \phi$
Group order	<code> G </code>	$ G $
Normal subgroup	<code>\trianglelefteq</code>	$N \trianglelefteq G$
Quotient	<code>G/H</code>	G/H
Lie algebra	<code>\mathfrak{g}</code>	$\mathfrak{g}$
Congruence mod m	<code>x \equiv b \pmod{m}</code>	$x \equiv b \pmod{m}$

The modular form compiles: `x \equiv b \pmod{m}` lowers to `Eq(Mod(x, m), b)`. Group notation (`\langle a \rangle`, `\circ`, `\star`, `\trianglelefteq`) is display-level — infix symbols, no algebraic structure attached.

Differential equations

You want	Type	Renders
First / second derivative	<code>y'</code> , <code>y''</code>	y', y''
Time derivative	<code>\dot{x}</code> <code>\ddot{x}</code>	$\dot{x}, \ddot{x}$
n th derivative	<code>f^{(n)}(x)</code>	$f^{(n)}(x)$
Leibniz derivative	<code>\frac{dy}{dx}</code>	$\frac{dy}{dx}$
Dependent variable	<code>\text{def} y(t)</code>	declares <code>y</code> a function of <code>t</code>
Laplace transform	<code>\mathcal{L}</code>	$\mathcal{L}\{f\}$ — notation only

Declaring `\text{def} y(t)` marks `y` as a dependent variable, so `y'`, `y''`, and `y^{(n)}` emit real `Derivative / diff` calls in `t`. Dotted names read as functions of `t` on their own — `\dot{x}` is `x(t)` differentiated once. `\mathcal{L}` is notation only.

Partial differential equations

You want	Type	Renders
Partial derivative	<code>\frac{\partial u}{\partial t}</code>	$\frac{\partial u}{\partial t}$
Partial derivative, shorter	<code>\partial_ t</code> then <code>u</code>	$\partial_t u$
Laplacian	<code>\nabla^2 u</code> or <code>\Delta u</code>	$\nabla^2 u, \Delta u$
Divergence / curl	<code>\nabla \cdot</code> / <code>\nabla \times</code>	$\nabla \cdot \vec{F}, \nabla \times \vec{F}$
Subscript notation	<code>u_t</code> , <code>u_{xx}</code>	u_t, u_{xx}

You want	Type	Renders
Condition at infinity	<code>\infty</code>	∞

`\partial_{t}`u compiles to the partial derivative, like the `\frac` form. The `\nabla`, `\Delta`, divergence, and curl spellings are display notation — no vector-calculus semantics behind them.

Combinatorics

You want	Type	Renders
Binomial coefficient	<code>\binom{n}{k}</code>	$\binom{n}{k}$ — <code>binomial(n, k)</code>
Factorial / double factorial	<code>n!</code> , <code>n!!</code>	$n!$, $n!!$
Floor / ceiling	<code>\lfloor x \rfloor</code> , <code>\lceil x \rceil</code>	$\lfloor x \rfloor$, $\lceil x \rceil$
Sum / product	<code>\sum</code> / <code>\prod</code> with bounds	$\sum_{k=0}^n$, $\prod_{k=1}^n$
Gamma function	<code>\Gamma(z)</code>	$\Gamma(z)$ — <code>gamma(z)</code>
Totient and friends	<code>\mathrm{totient}(n)</code>	$\mathrm{totient}(n)$
Ellipsis	<code>\dots</code> <code>\cdots</code>	$1, 2, \dots, n$

`\binom`, `!` / `!!`, floors, ceilings, and `\Gamma` compile to the matching SymPy calls. Other number-theory words work like `totient` via `\mathrm{name}` — `mobius`, `nextprime`, `factorint` — as long as the name is a plain word.

Statistics

The calculator speaks a set of `scipy.stats` calls, typed as upright words `\mathrm{name}(args)`. The first stats cell in a session loads the scipy engine on top of SymPy (about a second more before the row lands); cells without stats names aren't affected.

Distributions

A distribution call is `<name><method>` — `normcdf`, `binompmf`, `tppf`. Available distributions: `norm`, `t`, `chi2`, `expon`, `uniform`, `beta`, `gamma`, `lognorm`, `cauchy`, `f` (continuous) and `binom`, `nbinom`, `poisson`, `geom`, `hypergeom` (discrete). Methods: `pdf` / `pmf`, `cdf`, `ppf`, `sf`, `isf`, `mean`, `var`, `std`, `median`, `stats`, `moment`, `interval`, `rvs`, and `fit` (continuous only). Distribution parameters come after the point and can be left off to get scipy's defaults — `\mathrm{normcdf}(x)` is the standard normal.

You want	Type	Renders
Density of N(0,1) at x	<code>\mathrm{normpdf}(x)</code>	normpdf(x) → <code>st.norm.pdf(x)</code>
Normal CDF, mean μ , sd σ	<code>\mathrm{normcdf}(x, \mu, \sigma)</code>	normcdf(x, μ, σ) → <code>st.norm.cdf(x, mu, sigma)</code>
Inverse CDF (quantile)	<code>\mathrm{tppf}(0.975, 9)</code>	tppf(0.975, 9) → <code>st.t.ppf(0.975, 9)</code>
Upper-tail probability	<code>\mathrm{normsf}(2)</code>	normsf(2) → <code>st.norm.sf(2)</code>
Dist. mean / var / std	<code>\mathrm{binomvar}(10, 0.5)</code>	binomvar(10, 0.5) → <code>st.binom.var(10, 0.5)</code>

You want	Type	Renders
Dist. central interval	$\mathrm{tinterval}(0.95,9)$	tinterval(0.95,9) → <code>st.t.interval(0.95, 9)</code>
Random draws	$\mathrm{normrvs}(5)$	normrvs(5) → <code>st.norm.rvs(size=5)</code>
Fit a dist. to data	$\mathrm{normfit}([1,2,3,4])$	normfit(...) → <code>st.norm.fit(list(...))</code>

Sample statistics

The data argument is a list `[1,2,3]`, a set `{1,2,3}`, or a name the cell already bound (`d = [1,2,3]` then $\mathrm{ttest}(d,0)$).

You want	Type	Renders
Sample mean	$\mathrm{smean}(d)$	smean(d) → <code>np.mean(list(d))</code>
Median	$\mathrm{median}(d)$	median(d) → <code>np.median(list(d))</code>
Sample variance / sd	$\mathrm{svar}(d)$, $\mathrm{sstd}(d)$	svar(d) → <code>np.var(list(d), ddof=1)</code>
Std. error of the mean	$\mathrm{sem}(d)$	sem(d) → <code>st.sem(list(d))</code>
Skewness / kurtosis	$\mathrm{skew}(d)$, $\mathrm{kurtosis}(d)$	skew(d) → <code>st.skew(list(d))</code>
Interquartile range	$\mathrm{iqr}(d)$	iqr(d) → <code>st.iqr(list(d))</code>
Geometric / harmonic mean	$\mathrm{gmean}(d)$, $\mathrm{hmean}(d)$	gmean(d) → <code>st.gmean(list(d))</code>
Summary tuple	$\mathrm{describe}(d)$	describe(d) → <code>st.describe(list(d))</code>
Z-scores	$\mathrm{zscore}(d)$	zscore(d) → <code>st.zscore(list(d))</code>
One-sample t-test	$\mathrm{ttest}(d,0)$	ttest(d,0) → <code>st.ttest_1samp(list(d), 0)</code>
Conf. interval of mean	$\mathrm{meanconf}(d) / (d,0.99)$	meanconf(d) → <code>mc_mean_ci(d, 0.95)</code>

Comparing samples and hypothesis tests

Two-sample tests take two data operands; the multi-sample tests take two or more (`friedman` needs at least three). Everything prints as a `(statistic, p-value)` tuple unless noted.

You want	Type	Renders
Independent-samples t-test	$\mathrm{ttestind}(d,e)$	ttestind(d,e) → <code>st.ttest_ind(list(d), list(e))</code>
Paired t-test	$\mathrm{ttestrel}(d,e)$	ttestrel(d,e) → <code>st.ttest_rel(list(d), list(e))</code>
Mann-Whitney U	$\mathrm{mannwhitneyu}(d,e)$	mannwhitneyu(d,e) → <code>st.mannwhitneyu(list(d), list(e))</code>

You want	Type	Renders
Wilcoxon signed-rank	<code>\mathrm{wilcoxon}(d,e)</code>	wilcoxon (<i>d, e</i>) → <code>st.wilcoxon(list(d), list(e))</code>
Two-sample KS	<code>\mathrm{ks2samp}(d,e)</code>	ks2samp (<i>d, e</i>) → <code>st.ks_2samp(list(d), list(e))</code>
Correlations	<code>\mathrm{pearsonr}</code> / <code>\mathrm{spearmanr}</code> / <code>\mathrm{kendalltau}</code> (<i>x, y</i>)	pearsonr (<i>x, y</i>) → <code>st.pearsonr(list(x), list(y))</code>
Equal-variance tests	<code>\mathrm{levene}</code> / <code>\mathrm{bartlett}</code> / <code>\mathrm{fligner}</code> (<i>d, e</i>)	levene (<i>d, e</i>) → <code>st.levene(list(d), list(e))</code>
One-way ANOVA	<code>\mathrm{foneway}(d,e,...)</code>	foneway (<i>d, e</i>) → <code>st.f_oneway(list(d), list(e))</code>
Friedman test	<code>\mathrm{friedman}(d,e,f)</code>	friedman (<i>d, e, f</i>) → <code>st.friedmanchisquare(...)</code>
Normality tests	<code>\mathrm{shapiro}</code> / <code>\mathrm{normaltest}</code> / <code>\mathrm{jarquebera}</code> / <code>\mathrm{skewtest}</code> / <code>\mathrm{kurtosistest}</code> (<i>d</i>)	shapiro (<i>d</i>) → <code>st.shapiro(list(d))</code>
Fit against a distribution	<code>\mathrm{kstest}(d)</code> / <code>\mathrm{anderson}(d)</code>	kstest (<i>d</i>) → <code>st.kstest(list(d), 'norm')</code>
Chi-square goodness of fit	<code>\mathrm{chisquare}(obs,exp?)</code>	chisquare (<i>obs</i>) → <code>st.chisquare(list(obs))</code>
Binomial test	<code>\mathrm{binomtest}(k,n,p?)</code>	binomtest (<i>3, 10, 0.5</i>) → <code>st.binomtest(3, 10, 0.5)</code>

Worth knowing:

- `\mathrm{smean}` rather than `\mathrm{mean}` — `mean(x)` already prints the expectation symbol $\bar{x}$.
- Sample `svar` / `sstd` are the unbiased versions (`ddof=1`).
- `pdf` on a discrete distribution (or `pmf` on a continuous one) is an error — the message names the right method.
- `fit` exists only for the continuous distributions.
- `\mathrm{meanconf}` emits a small `mc_mean_ci` helper into the generating code — it wraps `st.t.interval` around the sample mean.
- `\mathrm{ttestind}` (independent) vs `\mathrm{ttestrel}` (paired) mirror scipy's `ttest_ind` / `ttest_rel`; `\mathrm{ttest}` stays the one-sample test.
- `\mathrm{kstest}` / `\mathrm{anderson}` test against `'norm'` by default; a second arg names another scipy distribution — `\mathrm{kstest}(d, \mathrm{expon})`.
- `\mathrm{chisquare}`'s expected counts must sum to the observed total (scipy rescales nothing).
- `\mathrm{anderson}` and `\mathrm{binomtest}` print their full result objects rather than a plain tuple.
- `\mathrm{bartlett}` float-coerces its samples in the emitted code — scipy raises on integer-typed data.
- Results are scipy result objects — the `statistic` / `pvalue` fields print as a pair; a NaN pair from a too-small sample is scipy's own answer, not a compile error.

Output targets

Pick a target from the **Output** menu in the header.

LaTeX

The output column shows each cell's serialized LaTeX; **Copy** puts it on the clipboard.

Calculator

Each cell evaluates through SymPy — a real computer-algebra system that runs in the page as WebAssembly. It takes a while to download and boot; until the engine is ready, results come from a lightweight JavaScript engine and are dimmed and marked *estimate*. The chip at the top shows engine status.

- Each statement in a cell gets a result row; $\approx$ adds a decimal approximation when one exists.
- **!** marks a statement that failed, **i** a note; hover for the message.
- **Show generating code** reveals the exact Python program that produced the results, with a copy button.
- The **Settings** menu (calculator and Python targets) exposes display plumbing and animation timing — default values are fine.

Python

The output column shows the SymPy program MathCompile generates per cell, syntax-highlighted. **Copy** grabs one cell's lines; **Copy script** grabs the whole worksheet as one runnable program. The `import *` checkbox controls the import style. Statements the compiler can't express appear as **! / i** notes under the code.

One rule to know: write `\text{def} f(x) = 2x` (type `def` in Smart mode) to define a function — the generated code is `def f(x): return 2*x`. A bare `f(x)` is read as multiplication, `f * x`.

Keyboard reference

Key	What it does
Enter	new line in the cell; accepts an open <code>\command</code> ; adds a row in a matrix
Shift+Enter	new cell below
Right , Tab	step out of the current block
Esc	dismiss the autocomplete menu or picker; returns focus to the field
Ctrl+Space	open the symbol picker
Ctrl+K / ⌘K	command palette
Shift+Space	new column inside a matrix
↑ , ↓	move between cells at field edges
Home , End	block edges (Ctrl+Home / Ctrl+End for the whole field)
Backspace on a blank cell	delete the cell

Saving

The worksheet, theme, Smart mode, and target choice persist in your browser's local storage — there's nothing to click. Clearing the site's data resets everything.

Repository docs: `README.md` (build/run), `SYMBOLS.md` (command inventory), `AGENTS.md` (architecture), `CALCULATOR-ENGINES.md` (calculator internals).